

Visual Product Search Engine

DeepFashion In-Shop Clothes Retrieval Project Report

IMT2023510 Shubhranil Basak
IMT2023051 Saheem Showkat Reshi

May 15, 2026

Abstract

This report documents a query-by-image visual product retrieval system built for the DeepFashion In-Shop Clothes Retrieval dataset. The system accepts a clothing image, localizes the target garment, embeds visual and semantic information, searches a vector index, and re-ranks candidates to return visually and semantically similar catalog products. The implemented pipeline combines YOLO clothing detection, CLIP embeddings, BLIP-2 captioning and image-text matching, and HNSW approximate nearest-neighbor indexing. The report also summarizes exploratory data analysis from the accompanying notebook, the training and indexing scripts, the Streamlit demo applications, the evaluation protocol, and the project repository index.

Contents

1	Project Links and Code Index	3
2	Problem Statement and Motivation	3
3	Dataset and Exploratory Data Analysis	3
3.1	Raw Annotation Files	4
3.2	Integrity Checks	4
3.3	Official Splits	4
3.4	Category and Gender Distribution	4
4	System Overview	6
4.1	Model Components	7
5	Offline Indexing Pipeline	7

6	Online Retrieval Pipeline	8
6.1	Query Upload and Model Loading	8
6.2	YOLO Detection and Cropping	8
6.3	Full-Body Bounding Box Construction	8
6.4	Query Encoding and Search Variants	9
6.5	BLIP-2 ITM Re-Ranking and Display	9
7	Training and Fine-Tuning Strategy	9
8	Ablation Study Design	10
9	Evaluation Protocol	10
9.1	Metric Function Definitions	10
9.2	Experimental Results	11
10	Streamlit Demonstration	12
11	Reproducible Usage	13
12	Conclusion	14

1 Project Links and Code Index

GitHub repository: [Black-Hawk21/DeepFashion-In-shop-Clothes-Retrieval](#)

File or asset	Purpose
scripts/train_clip.py	Fine-tunes CLIP for item-level retrieval so images of the same <code>item_id</code> are closer in embedding space.
scripts/build_index.py	Builds the offline gallery index by producing embeddings, storing metadata, and writing HNSW index files under <code>index/</code> .
scripts/evaluate.py	Runs batch retrieval evaluation on query images and reports Recall@K, NDCG@K, and mAP@K.
app/streamlit_best_demo.py	Best interactive demo: query crop, BLIP-2 query caption, CLIP image/text fusion, HNSW retrieval, and BLIP-2 ITM re-ranking.
notebooks/deepfashion_eda.ipynb	Exploratory data analysis notebook used to validate annotations, split counts, category distributions, and processed index tables.
README.md	Setup, model checkpoint, pipeline, ablation, and usage documentation.

2 Problem Statement and Motivation

Text-only product search often fails when users cannot describe a garment precisely or when sellers use inconsistent product titles and metadata. A shopper may have an image of a dress, shirt, skirt, hoodie, or pair of pants but may not know the exact category, fabric, fit, pattern, or color terms required by a keyword search engine. This project addresses that problem with **query-by-image retrieval**: the user uploads a product image, and the system returns catalog images that match the same or visually similar clothing item.

The project uses the DeepFashion In-Shop Clothes Retrieval dataset. The retrieval task treats two images as relevant when they share the same `item_id`. This makes the task realistic: the query and gallery images can contain the same product under different poses, crops, model views, lighting conditions, and backgrounds.

3 Dataset and Exploratory Data Analysis

The EDA notebook loads the DeepFashion annotation files, validates their integrity, summarizes the official splits, and creates cleaned train/query/gallery tables for downstream indexing and evaluation.

3.1 Raw Annotation Files

The notebook parsed the metadata files actually used by the retrieval pipeline, as listed in Table 2. The report intentionally excludes landmarks, item descriptions, item-list metadata, and attribute-name files because they were not used for model training, indexing, or evaluation.

Metadata file/table	Rows parsed
Evaluation split table	52,712
Bounding boxes	52,712

Table 2: DeepFashion metadata used by the retrieval pipeline.

3.2 Integrity Checks

The notebook compared the declared row counts with the parsed data and found exact agreement for the evaluation split table and bounding-box table. It also reported zero duplicate image names in both tables. Every evaluation image had a matching bounding-box row, and a random sample of 5,000 images contained no missing image files.

3.3 Official Splits

The official retrieval split contains 25,882 training images, 14,218 query images, and 12,612 gallery images. Query and gallery share 3,985 item identities, which is expected because query images must have matching products in the gallery. Train identities do not overlap with either query or gallery, preventing leakage between training and test retrieval.

Split	Images	Role
Train	25,882	Used to fine-tune CLIP.
Query	14,218	Used as retrieval inputs during evaluation.
Gallery	12,612	Search database indexed by HNSW.

Table 3: DeepFashion In-Shop retrieval split used by the project.

3.4 Category and Gender Distribution

EDA shows that the dataset is imbalanced toward women’s clothing. There are 44,874 women’s images and 7,838 men’s images. Unique item counts follow the same trend, with 7,053 women’s items and 929 men’s items. Clothing-type counts also differ by gender: women’s images include upper-body, lower-body, and full-body items, while the parsed men’s subset contains upper-body and lower-body examples but no full-body examples.

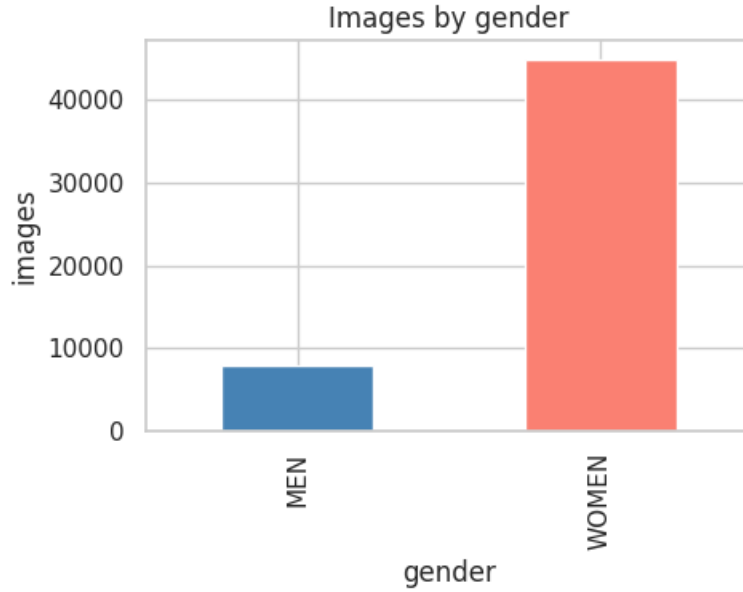


Figure 1: Image-count distribution by gender from the EDA notebook.

Gender	Upper-body	Lower-body	Full-body
MEN	5,397	2,441	0
WOMEN	28,090	8,053	8,731

Table 4: Clothing-type counts by gender from the EDA notebook.

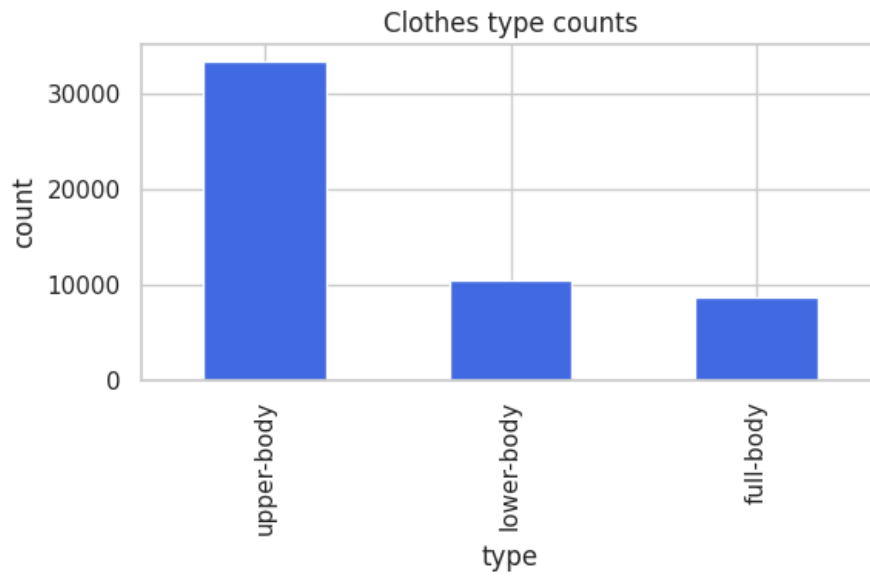


Figure 2: Overall clothing-type distribution: upper-body examples dominate the dataset, followed by lower-body and full-body garments.

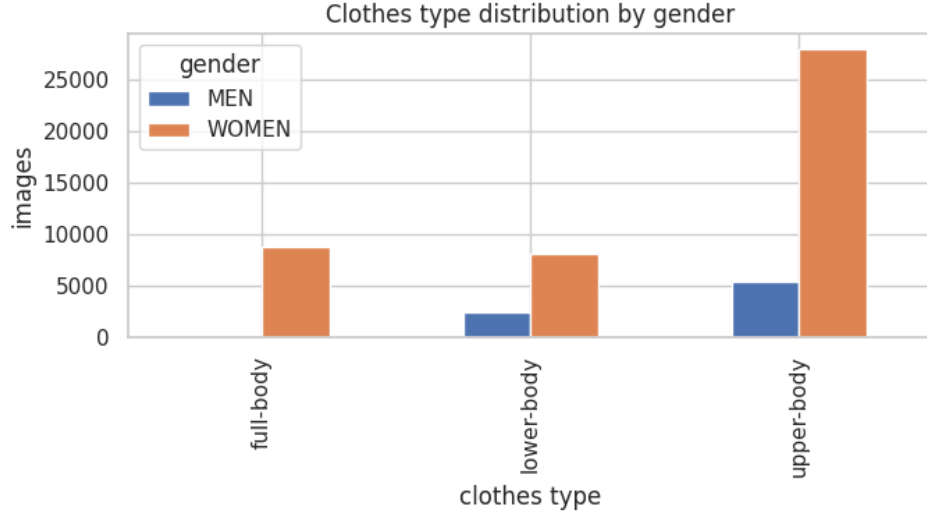


Figure 3: Clothing-type distribution by gender. Men’s examples are upper-body and lower-body only in the parsed split, while women’s examples span all three clothing types.

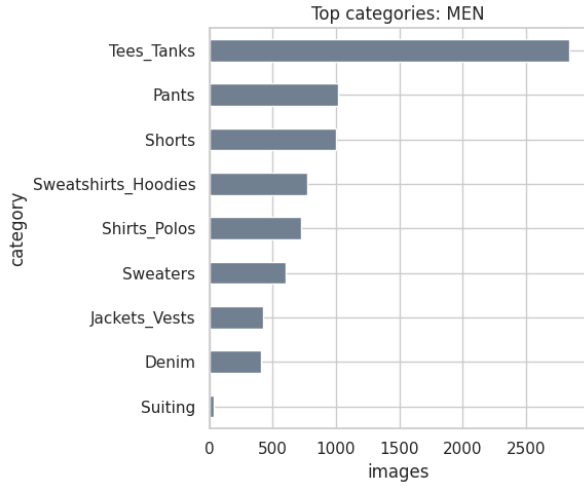


Figure 4: Top category counts for MEN.

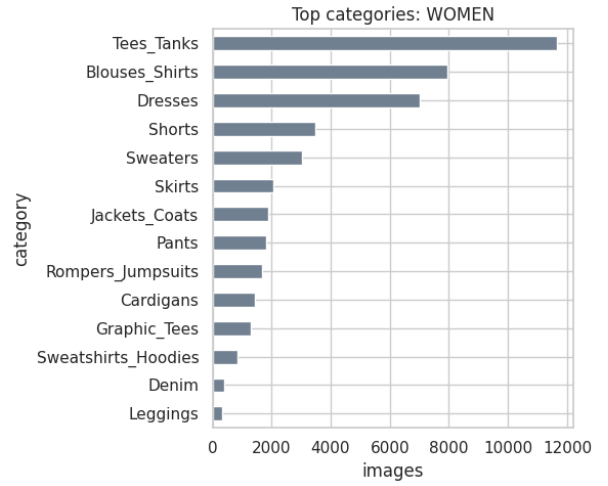
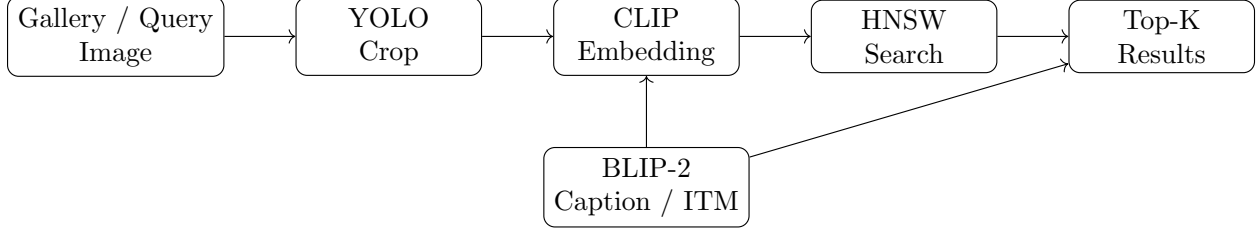


Figure 5: Top category counts for WOMEN.

4 System Overview

The system has two major phases: an offline indexing phase for gallery images and an online retrieval phase for user queries.



4.1 Model Components

- **YOLO clothing detector:** localizes the garment and crops the image to reduce background and model-pose noise. The detector was based on the `kesimeg/yolov8n-clothing-detection` YOLOv8 clothing-detection model, with the project checkpoint stored as `checkpoints/clothing_yolo.pt`.
- **CLIP:** encodes images and text into a shared embedding space. The project uses a fine-tuned checkpoint at `checkpoints/best_model.pt` for the best condition.
- **BLIP-2:** generates captions for gallery images and supplies image-text matching scores for re-ranking. The project uses `Salesforce/blip2-opt-2.7b`.
- **HNSW:** performs approximate nearest-neighbor search over normalized embedding vectors for efficient top-K candidate retrieval.

5 Offline Indexing Pipeline

For each gallery product image, the offline stage creates a searchable representation and stores it in the HNSW index.

1. **Load gallery metadata:** read the gallery split and item identifiers.
2. **Crop product region:** use YOLO or annotation boxes to focus on the main garment.
3. **Generate caption:** use BLIP-2 to produce a normalized semantic description for the image.
4. **Encode image and text:** compute CLIP visual embedding $\phi_V(\hat{x}_i)$ and caption embedding $\phi_T(c_i)$.
5. **Fuse modalities:** combine visual and textual features using an image weight α :

$$v_i = \alpha \phi_V(\hat{x}_i) + (1 - \alpha) \phi_T(c_i), \quad \|v_i\|_2 = 1. \quad (1)$$

6. **Build HNSW index:** insert normalized vectors and persist the index plus metadata under `index/`.

The indexing entry point is `scripts/build_index.py`. The README documents index artifacts such as `index/gallery_captions.json`, `index/Cond.A/`, `index/Cond.B/`, and `index/Cond.C/`.

6 Online Retrieval Pipeline

At inference time, the user uploads a query image through Streamlit. The online retrieval phase was implemented in the demo file `app/streamlit_best_demo.py`. The goal of this stage is to turn a raw user image into the same kind of normalized vector representation used by the offline HNSW gallery index, then re-rank retrieved candidates with a stronger vision-language model.

6.1 Query Upload and Model Loading

The Streamlit app first accepts a JPEG or PNG query image and converts it to RGB. It then loads the required runtime components once using cached resources: the fine-tuned CLIP checkpoint, the BLIP-2 processor/model, the YOLO detector, the HNSW index, gallery item metadata, and pre-computed gallery captions. This caching is important because CLIP, BLIP-2, and YOLO are heavy models and should not be reloaded every time the user presses the search button.

6.2 YOLO Detection and Cropping

The first online step is garment localization. The uploaded image is passed to the YOLO detector from `checkpoints/clothing_yolo.pt` using confidence threshold `YOLO_CONF = 0.25` and IoU threshold `YOLO_IOU = 0.45`. The app calls `yolo_model.predict` on the NumPy version of the PIL image and reads the predicted `xyxy` boxes. Only detections whose class name matches the clothing label set are retained, so non-clothing boxes are ignored.

Each accepted YOLO box is converted to integer coordinates (x_1, y_1, x_2, y_2) and then padded by 5% using `CROP_PADDING = 0.05`. Padding is applied relative to the detected box width and height, and the coordinates are clipped to the image boundaries. This avoids cutting off sleeves, hems, collars, or other edge details while still suppressing most background clutter. The app sorts detections by confidence, draws red boxes on the original image, and shows the user all proposed clothing crops.

6.3 Full-Body Bounding Box Construction

The full-body crop is not a separate YOLO prediction. It is constructed as a union box over all accepted clothing detections. If YOLO finds multiple garment regions, the app computes

$$x_1^{\text{union}} = \min_i x_{1,i}, \quad y_1^{\text{union}} = \min_i y_{1,i}, \quad x_2^{\text{union}} = \max_i x_{2,i}, \quad y_2^{\text{union}} = \max_i y_{2,i}.$$

The same 5% padding function is then applied to this union box, again clipped to image boundaries, and the resulting region is cropped from the original query image. This gives a full-body or full-outfit crop that contains all detected clothing regions together. It is useful when the query contains multiple garments or when outfit-level context is more helpful than a single item crop.

The interface displays both the individual YOLO crops and the union full-body crop. A radio-button selection lets the user explicitly choose the crop used for search. If YOLO finds no clothing boxes, the fallback is to use the full uploaded image as the query crop.

6.4 Query Encoding and Search Variants

After crop confirmation, `streamlit_best_demo.py` generates a BLIP-2 caption for the selected query crop. It then encodes both the crop and caption with CLIP and fuses them as $v_q = \alpha\phi_V(q) + (1 - \alpha)\phi_T(c_q)$, followed by L2 normalization. The Streamlit sidebar includes a **Fusion alpha** slider for this value before the HNSW search is run. Moving the slider changes the query vector itself: $\alpha = 1.0$ makes the search image-only, $\alpha = 0.0$ makes it caption-only, and intermediate values blend visual and textual evidence. In practice, the user should choose an α close to the one used when building the selected index, so the query embedding distribution matches the gallery embedding distribution.

For candidate retrieval, the app queries HNSW for a pool larger than the final output: `rerank_pool = top_k × rerank_factor`. HNSW returns nearest neighbors by cosine distance, which the app converts to cosine similarity as $1 - d$. Fetching an expanded pool gives BLIP-2 more candidates to re-order before the final top-K display.

6.5 BLIP-2 ITM Re-Ranking and Display

The gallery captions were generated offline and loaded from `index/gallery_captions.json`. For every HNSW candidate, the app retrieves the candidate caption and scores how well it matches the query crop using BLIP-2. Since the selected BLIP-2 model does not expose a separate ITM classification head in the demo, the code uses a yes/no prompt: “Does this image show <candidate caption>?” It compares the first-token probabilities of “yes” and “no” and uses the normalized yes probability as the ITM score.

Candidates are sorted by this ITM score in descending order, and the final top-K results are displayed with product images, item identifiers, cosine similarities, ITM scores, and captions when enabled. Thus, the online phase combines fast vector retrieval with a slower but more semantically precise re-ranking step.

The main demo entry point is [app/streamlit_best_demo.py](#).

7 Training and Fine-Tuning Strategy

The project keeps YOLO and BLIP-2 frozen and fine-tunes CLIP for the retrieval objective. The goal is to improve item-level discrimination: images sharing the same `item_id` should be close in embedding space, while different items should be separated. Fine-tuning focuses on the CLIP vision encoder, while the text encoder remains frozen for stable text semantics.

The training script is [scripts/train_clip.py](#) and is run with:

```
python scripts/train_clip.py --config configs/config.yaml
```

After training, the best checkpoint is saved as `checkpoints/best_model.pt` and reused for index construction, evaluation, and Streamlit demos.

8 Ablation Study Design

The repository defines three experimental conditions to isolate the contribution of semantic captions and CLIP fine-tuning.

ID	Configuration	α	Purpose
A	Vision-only CLIP	1.0	Baseline retrieval without caption fusion or fine-tuning.
B	Frozen CLIP + frozen BLIP-2	0.5, 0.7	Measures the effect of BLIP-2 semantic captions without CLIP fine-tuning.
C	Fine-tuned CLIP + frozen BLIP-2	0.5, 0.7	Measures the combined benefit of semantic fusion and item-level CLIP fine-tuning.

Table 5: Ablation conditions documented by the repository.

Results are reported as mean $\pm$ standard deviation over three random seeds: 105, 051, and 510.

9 Evaluation Protocol

Ground truth is defined by item identity: a gallery image is relevant to a query if both have the same `item_id`. Metrics are computed at $K \in \{5, 10, 15\}$.

Metric	Meaning
Recall@K	Fraction of queries with at least one correct gallery match in the top-K results.
NDCG@K	Position-aware ranking score that rewards placing relevant images earlier.
mAP@K	Mean average precision up to rank K, rewarding both retrieval and ordering of relevant items.

Table 6: Retrieval metrics used for project evaluation.

Batch evaluation is handled by [scripts/evaluate.py](#):

```
python scripts/evaluate.py --config configs/config.yaml --split query
```

9.1 Metric Function Definitions

The evaluation script computes Recall@K, NDCG@K, and mAP@K using the following helper functions:

```
def _recall_at_k(relevant, k):
```

```

return float(relevant[:k].any())

def _ndcg_at_k(relevant, k):
    gains = relevant[:k].astype(float)
    discounts = 1.0 / np.log2(np.arange(2, k + 2, dtype=float))
    dcg = float((gains * discounts).sum())
    n_relevant = int(relevant.sum())
    ideal_k = min(n_relevant, k)
    ideal_dcg = float((np.ones(ideal_k) * discounts[:ideal_k]).sum()) if ideal_k > 0 else 0.0
    return dcg / ideal_dcg if ideal_dcg > 0 else 0.0

def _ap_at_k(relevant, k):
    gains = relevant[:k].astype(float)
    if gains.sum() == 0:
        return 0.0
    cumsum = np.cumsum(gains)
    positions = np.arange(1, k + 1, dtype=float)
    precisions = cumsum / positions
    return float((precisions * gains).sum() / min(int(relevant.sum()), k))

```

9.2 Experimental Results

Tables 7, 8, and 9 report the retrieval performance of all three ablation conditions for $K \in \{5, 10, 15\}$. All values are reported as mean $\pm$ standard deviation over three random seeds (105, 051, 510). Bold values mark the best result in each column.

Cond.	Configuration	Recall@5	Recall@10	Recall@15
A	Vision-only, $\alpha=1.0$	0.4545 $\pm$ 0.0002	0.5239 $\pm$ 0.0001	0.5616 $\pm$ 0.0002
B	Frozen CLIP, $\alpha=0.5$	0.4477 $\pm$ 0.0001	0.5263 $\pm$ 0.0002	0.5717 $\pm$ 0.0002
B	Frozen CLIP, $\alpha=0.7$	0.4705 $\pm$ 0.0001	0.5448 $\pm$ 0.0001	0.5859 $\pm$ 0.0001
C	Fine-tuned CLIP, $\alpha=0.5$	0.6657 $\pm$ 0.0001	0.7246 $\pm$ 0.0001	0.7555 $\pm$ 0.0001
C	Fine-tuned CLIP, $\alpha=0.7$	0.6781 $\pm$ 0.0001	0.7330 $\pm$ 0.0001	0.7612 $\pm$ 0.0001

Table 7: Recall@ K results for the ablation conditions.

Cond.	Configuration	NDCG@5	NDCG@10	NDCG@15
A	Vision-only, $\alpha=1.0$	0.3322 $\pm$ 0.0002	0.3713 $\pm$ 0.0002	0.3935 $\pm$ 0.0002
B	Frozen CLIP, $\alpha=0.5$	0.3157 $\pm$ 0.0001	0.3577 $\pm$ 0.0001	0.3824 $\pm$ 0.0001
B	Frozen CLIP, $\alpha=0.7$	0.3428 $\pm$ 0.0001	0.3837 $\pm$ 0.0001	0.4073 $\pm$ 0.0001
C	Fine-tuned CLIP, $\alpha=0.5$	0.5008 $\pm$ 0.0001	0.5491 $\pm$ 0.0001	0.5783 $\pm$ 0.0001
C	Fine-tuned CLIP, $\alpha=0.7$	0.5172 $\pm$ 0.0001	0.5646 $\pm$ 0.0001	0.5925 $\pm$ 0.0001

Table 8: NDCG@ K results for the ablation conditions.

Cond.	Configuration	mAP@5	mAP@10	mAP@15
A	Vision-only, $\alpha=1.0$	0.2925 ± 0.0002	0.3143 ± 0.0002	0.3248 ± 0.0002
B	Frozen CLIP, $\alpha=0.5$	0.2733 ± 0.0001	0.2963 ± 0.0001	0.3076 ± 0.0001
B	Frozen CLIP, $\alpha=0.7$	0.3011 ± 0.0001	0.3238 ± 0.0001	0.3347 ± 0.0001
C	Fine-tuned CLIP, $\alpha=0.5$	0.4437 ± 0.0001	0.4742 ± 0.0001	0.4911 ± 0.0001
C	Fine-tuned CLIP, $\alpha=0.7$	0.4612 ± 0.0001	0.4913 ± 0.0001	0.5078 ± 0.0001

Table 9: mAP@ K results for the ablation conditions.

Effect of BLIP-2 captioning without fine-tuning (A vs. B). Comparing condition A (vision-only, $\alpha=1$) with condition B (frozen CLIP fused with BLIP-2 captions) shows a mixed but instructive picture. At $\alpha=0.5$, adding captions marginally *hurts* Recall@5 (0.448 vs. 0.455) and NDCG@5 (0.316 vs. 0.332), suggesting that equal image-text weighting dilutes the visual signal when the text encoder is not aligned to the fashion domain. Shifting weight toward the visual modality ($\alpha=0.7$) recovers and slightly surpasses condition A across all metrics (e.g., Recall@15: 0.586 vs. 0.562; mAP@15: 0.335 vs. 0.325), indicating that a modest caption contribution is beneficial when the image embedding carries the majority of the signal. Overall, the gains from captioning alone are limited, implying that semantic alignment requires fine-tuning to be effective.

Effect of CLIP fine-tuning (B vs. C). Fine-tuning CLIP on the in-shop retrieval objective (condition C) produces the most substantial improvement across all metrics and all values of K . At $\alpha=0.7$, Recall@10 increases from 0.545 (B) to 0.733 (C), a relative gain of $\approx 34.5\%$; NDCG@10 improves from 0.384 to 0.565 (+47.1% relative); and mAP@10 improves from 0.324 to 0.491 (+51.5% relative). These large gains confirm that the pre-trained CLIP embedding space is not naturally suited for fine-grained instance-level retrieval: contrastive fine-tuning on same-item pairs substantially tightens the embedding clusters for matching products.

Effect of image-text weighting α in condition C. Within condition C, $\alpha=0.7$ consistently outperforms $\alpha=0.5$ across all metrics and all K , though the margins are modest (e.g., Recall@15: 0.761 vs. 0.756; mAP@15: 0.508 vs. 0.491). This suggests that visual features remain the primary signal for in-shop clothes retrieval, while the BLIP-2 caption provides a complementary semantic correction that is most beneficial at moderate rather than equal weight.

Summary. The best overall configuration is **Condition C with $\alpha=0.7$** , achieving Recall@10 of 0.733, NDCG@10 of 0.565, and mAP@10 of 0.491. The ablation confirms that CLIP fine-tuning is the single most impactful component of the pipeline, and that caption fusion provides additional, though smaller, gains when image weight dominates the fused embedding.

10 Streamlit Demonstration

The interactive demo satisfies the project requirement for an end-to-end application. It accepts an input image, performs garment localization, displays the crop, retrieves similar gallery images, and

presents ranked results. The best-performing demo can be launched with:

```
streamlit run app/streamlit_best_demo.py
```

The demo uses the same deployed artifacts as the offline pipeline: `checkpoints/best_model.pt`, `checkpoints/clothing_yolo.pt`, `index/gallery_captions.json`, and condition-specific HNSW index files such as `index/Cond_C/hnsw_alpha0.5_seed51.bin`.

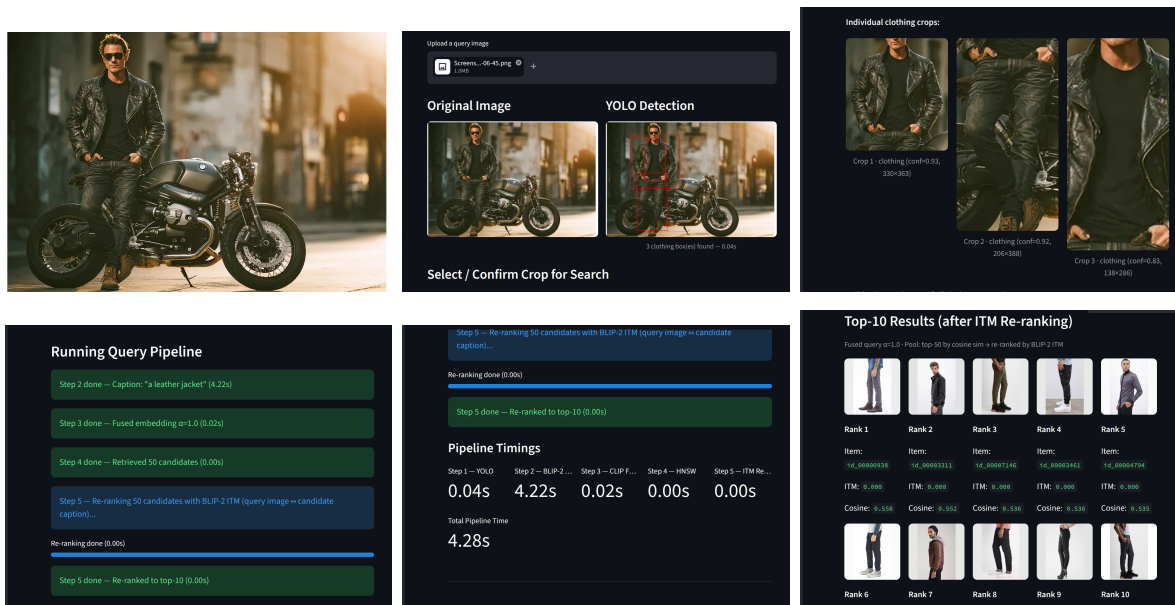


Figure 6: Compact Streamlit demo screenshots showing upload, YOLO crop confirmation, query processing, and retrieved product results.

11 Reproducible Usage

A typical end-to-end run follows the sequence below.

```
pip install -r requirements.txt
python scripts/train_clip.py --config configs/config.yaml
python scripts/build_index.py --config configs/config.yaml
python scripts/evaluate.py --config configs/config.yaml --split query
streamlit run app/streamlit_best_demo.py
```

The dataset should be placed under `data/` with annotation folders `data/Anno/`, `data/Eval/`, and image files under `data/Img/img/`. The EDA notebook can be run before training to regenerate processed metadata tables under `data/processed/`.

12 Conclusion

This project implements a complete visual product search system for DeepFashion In-Shop Clothes Retrieval. EDA confirms that the dataset splits and annotations are suitable for item-level retrieval, with clean query-gallery overlap and no train-test identity leakage. The retrieval pipeline combines the strengths of multiple models: YOLO improves localization, CLIP provides cross-modal embeddings, BLIP-2 adds semantic descriptions and image-text re-ranking, and HNSW enables fast nearest-neighbor search. The ablation design compares vision-only retrieval, frozen caption fusion, and fine-tuned CLIP fusion, making it possible to measure the contribution of each component fairly. The repository also includes practical scripts for training, indexing, evaluation, ablation, and Streamlit deployment.